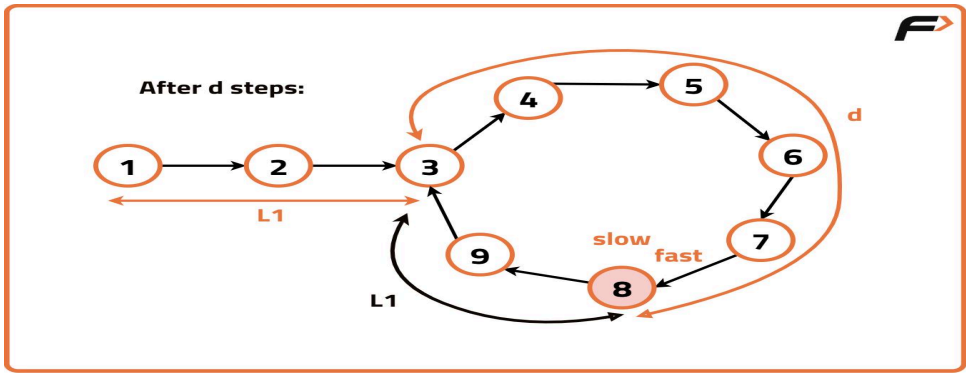
Find Cycle in Linked List

A cycle exists when a node points back to a previously visited node instead of pointing to `NULL`.



We use Floyd's Cycle Detection Algorithm, also called the Slow and Fast Pointer Approach.

- Slow pointer moves one step at a time.
- Fast pointer moves two steps at a time.
- If a cycle exists, they will eventually meet.

```C/C++
bool hasCycle(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast) {
            return true;
        }
    }

    return false;
}
```

Complexities:

- Time: O(n) Space: O(1)

Find Length of Loop

Once slow and fast meet, we know that a loop exists.

To calculate its length:

1. Keep one pointer at the meeting node.
2. Move another pointer around the loop.
3. Count the steps until it reaches the meeting node again.

```
None
int countLoopLength(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast) {
            int count = 1;
            Node* temp = slow->next;

            while (temp != slow) {
                count++;
                temp = temp->next;
            }

            return count;
        }
    }

    return 0;
}
```

Time: O(n)

Space: O(1)

6. Find Middle Element

We again use the Slow and Fast Pointer Approach.

- Slow moves one step.
- Fast moves two steps.

When fast reaches the end, slow will be at the middle.

```
None
Node* findMiddle(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    return slow;
}
```

Example:

`10 → 20 → 30 → 40 → 50`

Slow reaches `30`, so middle = 30.

Important: For an even-sized list, this implementation returns the second middle node.

Time: O(n)

Space: O(1)

7. Kth Node from End

We use two pointers: `fast` and `slow`.

Approach:

1. Move fast pointer K steps ahead.
2. Start moving both pointers together.
3. When fast reaches NULL, slow points to the Kth node from the end.

```
None
Node* kthFromEnd(Node* head, int k) {
    Node* fast = head;
    Node* slow = head;

    for (int i = 0; i < k; i++) {
        if (fast == NULL) return NULL;
        fast = fast->next;
    }

    while (fast != NULL) {
        slow = slow->next;
        fast = fast->next;
    }

    return slow;
}
```

Example:

`10 → 20 → 30 → 40 → 50`

For K = 2:

Answer = `40`.

Time: O(n)

Space: O(1)

8. Reverse a Linked List

This is one of the most important linked-list interview questions.

We use three pointers:

- `prev` – stores the previous node.
- `curr` – stores the current node.
- `nextNode` – saves the next node before changing the link.

Example:

Before:

```
10 → 20 → 30 → NULL
```

After:

```
30 → 20 → 10 → NULL
```

Code

```
None
Node* reverseList(Node* head) {
    Node* prev = NULL;
    Node* curr = head;

    while (curr != NULL) {
        Node* nextNode = curr->next;

        curr->next = prev;

        prev = curr;
        curr = nextNode;
    }

    return prev;
}
```

Dry run

Step	prev	curr	nextNode
Initial	NULL	10	—
1	10	20	20
2	20	30	30
3	30	NULL	NULL

Final linked list:

```
30 → 20 → 10 → NULL
```

Time: O(n)

Space: O(1)